

Containers 详细阐述

一、Containers 核心概念

在 NiceGUI 中，**Containers (容器)** 是用于组织和布局其他 UI 元素（如按钮、文本、输入框等）的基础组件。你可以把它们理解为「装东西的盒子」，这些盒子不仅能容纳其他组件，还能控制内部组件的排列方式、样式、可见性，甚至实现动态更新和响应式布局。

容器的核心价值：

- 结构化：将零散的 UI 元素分组，让界面逻辑更清晰；
- 布局控制：通过容器的属性调整内部元素的排列（横向 / 纵向、对齐方式、间距等）；
- 样式复用：给容器设置样式（如背景、边框），内部元素可继承或基于容器做样式适配；
- 批量控制：可通过控制容器的显隐、禁用状态，批量管理内部所有元素。

二、NiceGUI 中常用的 Containers 类型

NiceGUI 提供了多种内置容器，每种适用于不同的场景，以下是最常用的类型及详细用法：

1. 基础容器： `ui.element()`（通用容器）

这是所有容器的「基类」，也是最灵活的通用容器，本质是一个 HTML 元素（默认是 `<div>`），可自定义标签和任意属性。

用法示例

```
from nicegui import ui

# 创建一个通用容器（默认<div>），设置样式
with ui.element().style('border: 1px solid #ccc; padding: 20px; border-radius: 8px;'):
    ui.label('这是通用容器内的文本')
    ui.button('容器内的按钮')

# 自定义 HTML 标签（如<section>）
with ui.element('section').style('background-color: #f5f5f5; padding: 15px;'):
    ui.input('自定义标签容器内的输入框')

ui.run()
```

核心特点

- 无预设布局，完全自定义；
- 可通过 `style()` 或 `classes()` 控制样式；
- 支持所有 HTML 元素的属性（如 `id`、`name`）。

2. 布局容器： `ui.row()` & `ui.column()`

这是最常用的布局容器，专门用于控制内部元素的排列方向：

- `ui.row()`： **横向排列**（行），内部元素从左到右依次排列；
- `ui.column()`： **纵向排列**（列），内部元素从上到下依次排列。

用法示例

```
from nicegui import ui

# 外层列容器（整体纵向布局）
with ui.column().style('gap: 20px; padding: 20px;'):
    ui.label('基础布局示例')

    # 行容器（横向排列按钮）
    with ui.row().style('gap: 10px;'):
        ui.button('按钮1').style('background-color: #42b983;')
        ui.button('按钮2').style('background-color: #3498db;')
        ui.button('按钮3').style('background-color: #e74c3c;')

    # 列容器（纵向排列输入框）
    with ui.column().style('gap: 8px; border: 1px solid #eee; padding: 10px;'):
        ui.input('用户名').label('用户名')
        ui.input('密码').label('密码').password()
        ui.button('提交').style('width: 100%;')

ui.run()
```

核心属性（布局控制）

属性 / 方法	作用
<code>style('gap: 10px')</code>	控制内部元素的间距（行 / 列容器最常用）
<code>align_items()</code>	对齐方式（如 <code>ui.row().align_items('center')</code> 让元素垂直居中）
<code>justify_content()</code>	分布方式（如 <code>ui.row().justify_content('space-between')</code> 元素两端对齐）

3. 卡片容器： `ui.card()`

用于创建带阴影、圆角的卡片式容器，适合展示独立模块（如数据卡片、功能卡片），是美化界面的常用容器。

用法示例

```
from nicegui import ui

# 一行展示多个卡片
with ui.row().style('gap: 20px; padding: 20px; flex-wrap: wrap;'):
    # 卡片1: 基础信息
    with ui.card().style('width: 300px; padding: 15px;'):
```

```
        ui.label('用户信息').style('font-size: 18px; font-weight: bold; margin-bottom: 10px;')
        ui.label('姓名: 张三')
        ui.label('年龄: 25')
        ui.label('邮箱: zhangsan@example.com')

# 卡片2: 数据统计
with ui.card().style('width: 300px; padding: 15px;'):
    ui.label('今日数据').style('font-size: 18px; font-weight: bold; margin-bottom: 10px;')
    ui.label('访问量: 1258').style('font-size: 24px; color: #42b983;')
    ui.label('转化率: 18.5%').style('font-size: 24px; color: #3498db;')

ui.run()
```

核心特点

- 自带阴影（`box-shadow`）和圆角（`border-radius`）；
- 可通过 `style()` 自定义宽度、背景、内边距；
- 适合组合 `ui.row()` / `ui.column()` 实现卡片网格布局。

4. 滚动容器: `ui.scroll_area()`

当内部内容超出容器尺寸时，自动生成滚动条，适合展示长文本、长列表等内容。

用法示例

```
from nicegui import ui

# 创建固定高度的滚动容器
with ui.scroll_area().style('height: 200px; width: 400px; border: 1px solid #ccc;'):
    # 填充大量文本
    for i in range(20):
        ui.label(f'这是滚动容器内的第 {i+1} 行文本').style('margin: 5px 0;')

ui.run()
```

核心属性

- `style('height: 200px')`：必须设置固定高度（或宽度），否则不会触发滚动；
- 支持横向滚动：`style('white-space: nowrap; overflow-x: auto;')`（配合横向内容）。

5. 选项卡容器: `ui.tabs()` + `ui.tab_panel()`

属于「组合式容器」，用于创建多标签页布局，不同标签页对应不同的面板容器，适合分类展示内容。

用法示例

```
from nicegui import ui

# 创建选项卡
tabs = ui.tabs([( '首页', 'home'), ( '设置', 'settings'), ( '关于', 'about')])

# 选项卡面板容器
with ui.tab_panels(tabs, value='home').style('width: 400px; min-height: 200px; border: 1px solid #eee; padding: 15px;'):
    # 首页面板
    with ui.tab_panel('home'):
        ui.label('欢迎来到首页! ')
        ui.button('首页按钮')

    # 设置面板
    with ui.tab_panel('settings'):
        ui.label('设置页面')
        ui.checkbox('开启通知')
        ui.slider(label='音量', min=0, max=100)

    # 关于面板
    with ui.tab_panel('about'):
        ui.label('NiceGUI 容器示例 v1.0')

ui.run()
```

6. 其他常用容器

容器类型	作用
<code>ui.expansion()</code>	可折叠 / 展开的容器（手风琴效果），适合收纳次要内容
<code>ui.dialog()</code>	弹窗容器，用于展示临时内容（如确认框、表单）
<code>ui.drawer()</code>	侧边栏容器，可滑入 / 滑出，适合导航菜单
<code>ui.grid()</code>	网格布局容器，按行列数自动排列元素（适合响应式布局）

三、容器的核心操作（通用特性）

所有容器都支持以下通用操作，是使用容器的核心技巧：

1. 动态控制显隐

```
from nicegui import ui

# 创建容器并赋值给变量
container = ui.column().style('padding: 20px; border: 1px solid #ccc;')
with container:
    ui.label('可隐藏的容器内容')
    ui.button('内部按钮')

# 控制显隐的按钮
ui.button('显示/隐藏容器', on_click=lambda: container.toggle_visibility())

ui.run()
```

2. 动态添加 / 删除元素

```
from nicegui import ui

# 创建空容器
container = ui.row().style('gap: 10px; padding: 20px;')
count = 0

# 添加元素
def add_item():
    global count
    count += 1
    # 向容器中动态添加按钮
    btn = ui.button(f'动态按钮 {count}').parent(container)
    # 给按钮绑定删除自身的事件
    btn.on_click(lambda e: e.sender.delete())

ui.button('添加按钮', on_click=add_item)
ui.button('清空容器', on_click=lambda: container.clear())

ui.run()
```

3. 样式与类名控制

- `style()`：行内样式（优先级高，适合临时调整）；
- `classes()`：绑定 CSS 类名（适合复用样式，推荐）。

```
from nicegui import ui

# 定义全局 CSS 类
ui.add_css('''
    .custom-container {
        background-color: #f0f8ff;
        border: 2px solid #42b983;
        border-radius: 8px;
    }
''')
```

```
        padding: 15px;
    }
'''

# 使用类名
with ui.column().classes('custom-container'):
    ui.label('使用自定义类名的容器')
    # 叠加行内样式
    ui.button('按钮').style('color: white; background-color: #42b983;')

ui.run()
```

四、容器的嵌套与最佳实践

1. **嵌套原则**：容器可以无限嵌套，但建议控制层级（≤3 层），避免逻辑混乱；
2. **布局优先级**：先用 `ui.row()` / `ui.column()` 搭整体框架，再用 `ui.card()` / `ui.element()` 做局部美化；
3. **性能优化**：大量动态更新的内容建议用 `ui.scroll_area()` 包裹，避免整个页面重绘；
4. **响应式适配**：结合 CSS 的 `flex/grid` 和容器的 `style('width: 100%;')` 实现适配不同屏幕。

总结

1. NiceGUI 的 Containers 是组织 UI 元素的核心，核心作用是**布局控制、结构化、样式管理**；
2. 最常用的容器：`ui.row()` / `ui.column()`（布局）、`ui.card()`（美化）、`ui.scroll_area()`（滚动）；
3. 容器支持**动态显隐、动态增删元素、样式自定义**，是实现交互的基础；
4. 使用容器的关键是「先搭骨架（row/column），再填内容，最后美化（card/style）」。

掌握这些容器的用法，你就能轻松搭建出结构清晰、美观且交互性强的 NiceGUI 界面了。如果有具体的布局场景想实现，可以告诉我，我会针对性给出示例。

Clear Containers

To remove all elements from a row, column or card container, use can call `container.clear()`

Alternatively, you can remove individual elements by calling

- `container.remove(element: Element)`,
- `container.remove(index: int)`, or
- `element.delete()`.

```
from nicegui import ui

container = ui.row()

def add_face():
    with container:
        ui.icon('face')
add_face()
```

```
ui.button('Add', on_click=add_face)
ui.button('Remove', on_click=lambda: container.remove(0) if list(container) else None)
ui.button('Clear', on_click=container.clear)

ui.run()
```